

System Modules Breakdown: Secure-FEPRH Project

Based on the architecture of the **Secure-FEPRH (Faculty of Engineering Project & Research Hub)**, the system encompasses advanced backend security, real-time collaboration, artificial intelligence, and a modern React frontend.

The implementation of the system is divided into four distinct technical modules, designed to be handled in parallel.



Module 1: Core Architecture & Security

Focus: System Foundation, Cryptography, Multi-Tenancy, and DevOps

This module handles the most critical and complex infrastructure that the rest of the application relies upon. It requires deep knowledge of backend systems, security protocols, and database management.

Key Components to Implement:

1. Database Architecture & Multi-Tenancy:

- Design the SQLAlchemy database models (`app/models.py`).
- Implement the hierarchical data structure (Ministry → University → Faculty → Department).
- Manage database migrations using Alembic (`alembic/` , `alembic.ini`).

5. Authentication & Hierarchical RBAC (Role-Based Access Control):

- Implement secure JWT generation and validation (`app/auth.py`).
- Build strict dependency injection bouncers in FastAPI (`app/dependencies.py`) to ensure users can only access data within their clearance level (e.g., a Dean can only see their Faculty's projects).

8. Secure Document Repository:

- Implement AES-256-GCM encryption/decryption for all uploaded files (`app/crypto.py`).

10. Build the `file_validator.py` to perform magic byte analysis, blocking malicious executables before they hit the database.
11. **DevOps & Asynchronous Processing:**
12. Configure the Celery and Redis worker pipeline (`app/worker/celery_app.py`) to offload heavy background tasks.
13. Handle server deployment and local environment configurations.



Module 2: Artificial Intelligence & Biometrics Engine

Focus: Computer Vision, Natural Language Processing, and Python Data Science

This module focuses exclusively on the AI micro-services that give the application its cutting-edge "smart" capabilities.

Key Components to Implement:

1. **National Plagiarism Engine:**
2. **Code Similarity (`app/ai/ast_similarity.py`):** Write algorithms to parse Python/JS code into Abstract Syntax Trees (AST), strip out variable names, and calculate structural distances to catch students who rename variables.
3. **Text Similarity (`app/ai/nlp_similarity.py`):** Implement NLP pipelines using TF-IDF and Cosine Similarity to compare abstracts and project descriptions.
4. **Face Recognition Attendance (`app/ai/face_engine.py`):**
5. Integrate OpenCV and face recognition libraries.
6. Implement logic to convert webcam feeds into 128-dimensional facial encodings.
7. Write the distance comparison algorithm to verify student identities against their database encodings.
8. **AI Backend Routes:**
9. Expose the AI engines to the frontend via FastAPI endpoints (e.g., `/plagiarism/scan` and `/attendance/verify`).



Module 3: Real-Time Collaboration & Analytics

Focus: WebSockets, Dynamic Dashboards, and Agile Workflows

This module bridges the gap between the frontend and backend, focusing on real-time data flow, complex SQL aggregations, and interactive UI components.

Key Components to Implement:

1. **Real-Time Kanban Board:**
2. **Backend:** Implement WebSocket connections (`app/ws.py` , `app/routers/kanban.py`) to broadcast task movements to all connected clients instantly.
3. **Frontend:** Build the interactive drag-and-drop Kanban Board UI (`KanbanBoard.jsx`) using drag-and-drop libraries.
4. **Hierarchical Analytics Engine:**
5. **Backend:** Write complex SQLAlchemy queries (`app/routers/analytics.py`) that dynamically calculate statistics based on the requesting user's hierarchical role.
6. **Frontend:** Build the Command Center Dashboard (`DashboardHome.jsx`), integrating charting libraries to display the analytics visually.
7. **Meeting Management:**
8. Build the CRUD operations and frontend interface (`MeetingManagement.jsx`) for scheduling supervisor-student meetings and linking them to the AI attendance logs.



Module 4: Frontend Foundation, UX/UI, & User Management

Focus: React/Vite Infrastructure, Styling, State Management, and Core Portals

This module ensures the application looks professional, functions smoothly across devices, and provides intuitive portals for all user types.

Key Components to Implement:

1. **Global Design System & Theming:**
2. Configure Tailwind CSS (`tailwind.config.js`) and global CSS variables (`index.css`) to establish the aesthetic (dark modes, glassmorphism, primary color tokens).
3. Implement global layouts (`DashboardLayout.jsx` , `Sidebar` , `Topbar`) and route protection (`App.jsx`).
4. **Authentication Flow UI:**
5. Build the login and registration pages (`Login.jsx` , `Register.jsx`) with animations.

6. Manage global frontend state for user sessions via React Context (`AuthContext.jsx`) and Axios interceptors (`api.js`).

7. User Management & Profiles:

8. Build the Admin-level User Management interface (`UserManagement.jsx`) for creating and managing university staff and students.

9. Develop the User Profile page (`UserProfile.jsx`) handling secure password changes and avatar uploads.

10. Project Directory:

11. Build the main Projects List view (`ProjectsList.jsx`) with advanced filtering, searching, and pagination capabilities.

12. Create the Project Overview portal (`ProjectDetail.jsx`).